

Projeto Detalhado de Software

Princípios de Projeto Orientado a Objeto
(OOD – Object Oriented Design)

Informações

- **REFERÊNCIAS**

- **Princípios, padrões e práticas ágeis em C#**. MARTIN, Robert. MARTIN, Micah. 1ª edição. Capítulos 7, 8 e 9.
- “O conceito de Aberto e Fechado”, Microsoft
<http://msdn.microsoft.com/pt-br/magazine/cc546578.aspx>

- **LABORATÓRIOS**

- <a ser divulgado>

Motivação

- **Software se degrada**
 - Quando requisitos mudam de uma forma não antecipada
 - Equipe atual não entende a filosofia inicial do *design*
 - Falta de tempo ou habilidade para evoluir o *design*
- **Mas sabemos que...**
 - ... requisitos mudam
 - ... equipes mudam
 - ... prazo e custo são importantes para os negócios
 - Como evitar?

Maus Cheiros de Projetos

- Rigidez: software difícil de alterar. Mesmo pequenas mudanças geram uma sucessão de alterações em módulos dependentes. As mudanças podem ser mais difíceis do que seria esperado inicialmente
- Fragilidade: o software tem tendência a estragar em muitos lugares quando uma única alteração é feita. Os problemas podem estar em áreas que não tem relação conceitual com a mudança

Maus Cheiros de Projetos

- Imobilidade: quando o software contém partes que seriam úteis em outros sistemas, mas o trabalho e o risco envolvidos na separação dessas partes são grandes demais.
- Viscosidade: Projeto difícil de preservar. Existem formas de fazer alterações. Algumas preservam o projeto, outras não (ruim). Quando as mudanças malfeitas são mais fáceis de fazer que as corretas, a viscosidade do projeto é alta.

Maus Cheiros de Projetos

- Complexidade desnecessária: Contém elementos que não são úteis agora no projeto. Acontece por antecipação desnecessária de mudanças futuras.
- Repetição desnecessária: códigos repetidos
- Opacidade: Dificuldade de compreensão do módulo. Quando o sistema evolui é um risco, mas um esforço para manter o código claro e inteligível é necessário

Catálogo de Padrões/Princípios

- **GRASP**

- Conceituação e modelagem
- Como (distribuir) atribuir responsabilidades
- Padrões básicos de acoplamento, coesão, abstração e tratamento da variabilidade

- **PPOO**

- Voltado para o gerenciamento das dependências do *design*
- Induzem a evolução futura de forma mais adequada
- Gestão de dependências leva a código:
 - Fácil de evoluir (aceitar mudanças)
 - Robusto (mudanças sem grandes impactos)
 - Reusável

Princípios de Projeto OO

- **Princípios de Projeto OO (Principles of OOD)**
 - Princípios organizados por Robert C. Martin (Uncle Bob)
 - Foco no gerenciamento das dependências

PROJETO DE CLASSE

[S] Responsabilidade Única

[O] Aberto Fechado

[L] Substituição de Liskov

[I] Inversão de Dependência

[D] Segregação de Interface

PROJETO DE MÓDULO

Equivalência de Entrega/Reuso

Fechamento Comum

Reuso Comum

Dependências Acíclicas

Dependências Estáveis

Abstrações Estáveis

“Nunca deveria existir mais de uma razão para uma classe mudar.”

PPOO – Responsabilidade Única

- **Razão para Mudar == Responsabilidade da Classe**
 - Responsabilidades são do tipo “Fazer” e/ou “Saber”
 - Saber: uma informação pode mudar de formato ou ser desmembrada ou expandida
 - Fazer: uma nova forma de processar dados pode ser necessária
- **Cada responsabilidade é um eixo de mudança**
 - N responsabilidades, N motivos para mudar
 - Mudança pode levar a efeitos indesejáveis em “outras” responsabilidades próximas

PPOO – Responsabilidade Única

- **Induzir classes coesas**

- Facilita evolução e manutenção
- Serve como bom exemplo dentro da base de código

- **Exemplo**

- Situação 1: comportamentos A e B implementados em única classe concreta, com chaveamento interno
- Situação 2: comportamentos A e B implementados em classes concretas distintas e com interface comum
- Em caso de novo comportamento C, o que seria mais provável ocorrer em cada situação?

PPOO – Responsabilidade Única

```
public class MinhaCliente{  
    void fazer(int comportamentoDesejado){  
        if(comportamentoDesejado==1){  
            this.comportamentoA();  
        } else if(comportamentoDesejado==2){  
            this.comportamentoB;  
        } else if(comportamentoDesejado==3){  
            this.comportamentoC;  
        }  
    }  
    void comportamentoA(){}  
    void comportamentoB(){}  
    void comportamentoC(){}  
}
```

PPOO – Responsabilidade Única

```
public class MinhaCliente{  
    void fazer(FactiveI factivel){ factivel.fazer();}  
}  
  
public interface FactiveI{  
    void fazer();  
}  
  
public class ComportamentoA implements FactiveI{  
    void fazer();  
}  
  
public class ComportamentoB implements FactiveI{  
    void fazer();  
}  
  
public class ComportamentoC implements FactiveI{  
    void fazer();  
}
```

PPOO – Responsabilidade Única

- Exemplo de **violação** do princípio

```
public Class Conexao {  
  
    // estabelecimento/encerramento  
    public void iniciar();  
    public void finalizar();  
  
    // envio/recebimento  
    public void enviar(Comando);  
    public Comando receber();  
}
```

PPOO – Responsabilidade Única

- Bom exemplo do princípio

```
public Class GerenciadorConexao {  
  
    public CanalTransmissao iniciar();  
    public void finalizar();  
}
```

```
public Class CanalTransmissao {  
  
    public void enviar(Comando);  
    public Comando receber();  
}
```

PPOO – Responsabilidade Única

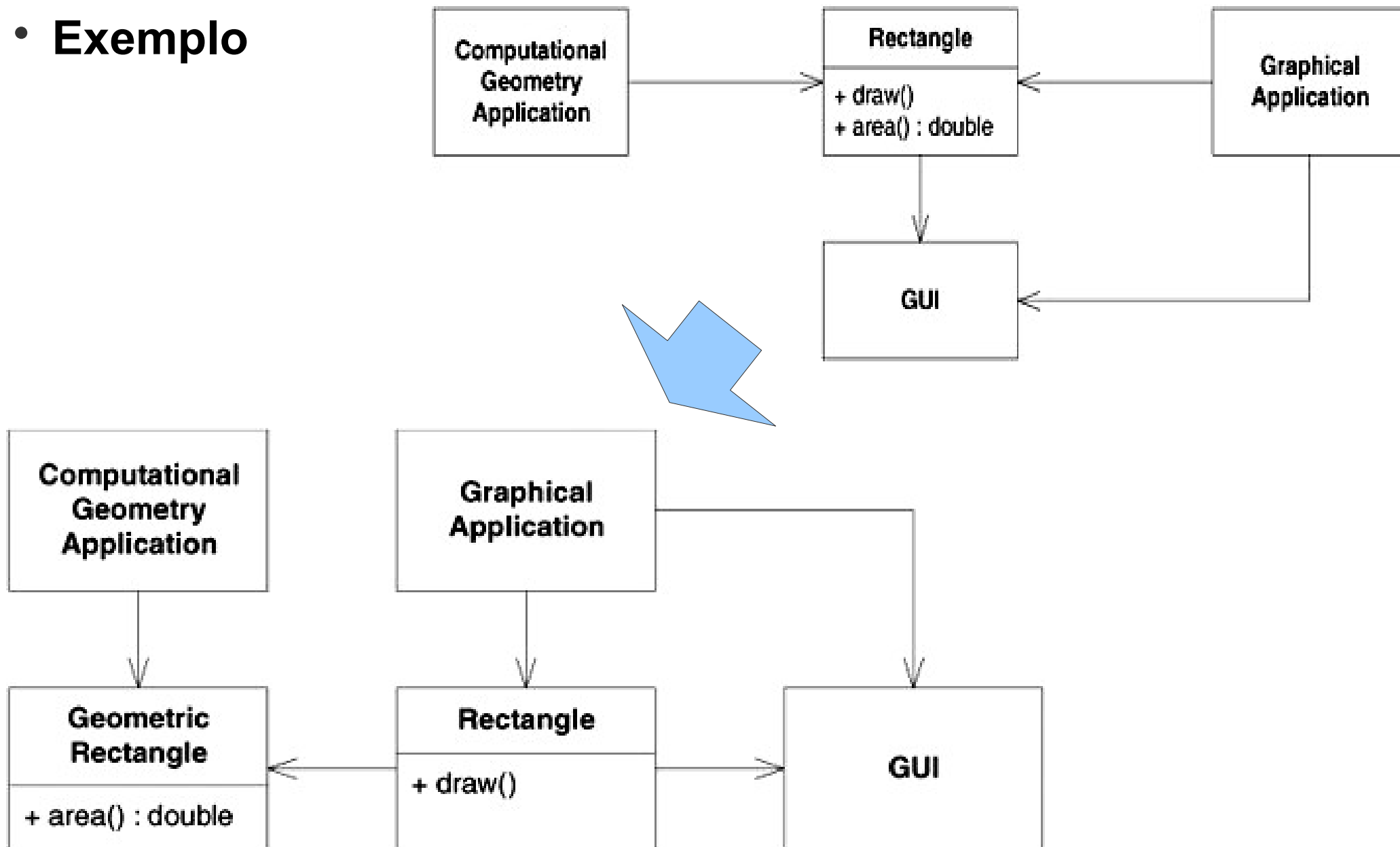
- **Separando responsabilidades (alguns exemplos)**
 - Estrutura de dados com múltiplas entidades de negócio
 - Solução: quebrar em agregações/composições
 - Algoritmos distintos para mesma estrutura de dados
 - Solução: separar algoritmos em classes distintas, atuando sobre mesma classe de dados
 - Algoritmos/comportamento complementar junto principal
 - Solução: separar comportamento complementar em classe distinta e estabelecer associação

PPOO – Responsabilidade Única

- **Single Responsibility Principle (SRP)**
- Princípio simples...
... porém difícil de fazer corretamente
- Identificar e separar responsabilidades é a essência básica de projetar software
- Os demais princípios referenciam o SRP diversas vezes

PPOO – Responsabilidade Única

- Exemplo



“Entidades de Software (Classes, Módulos, Funções) devem ser ABERTAS para extensão, mas FECHADAS para modificação.”

PPOO – Aberto-Fechado

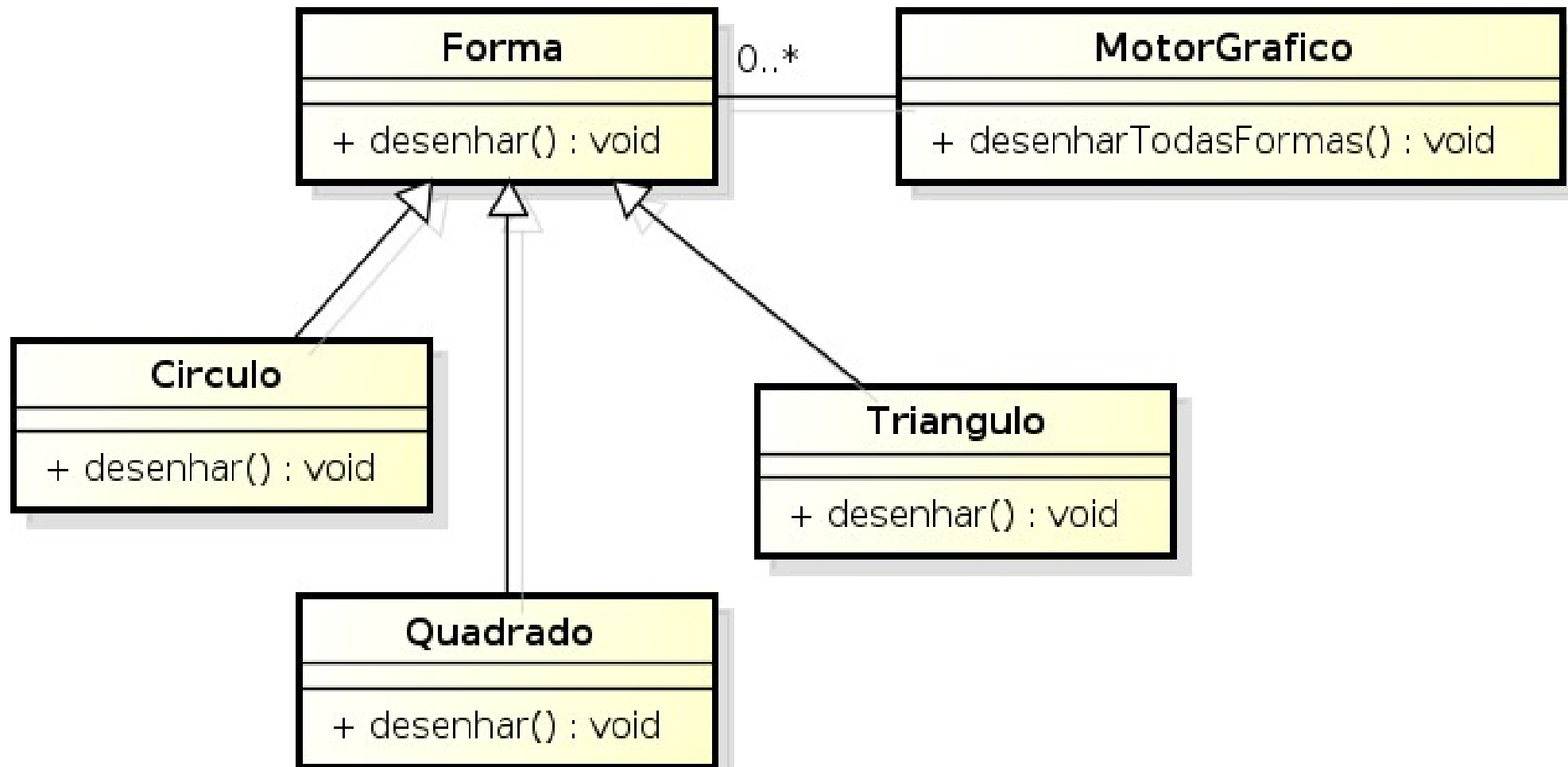
- **Aberto para extensão**
 - Comportamento do módulo pode ser estendido com novos comportamentos necessários para mudanças de requisitos
- **Fechado para modificação**
 - Estender comportamento não resulta em mudanças no código atual do módulo
- **Mudar sem Modificar... como é que é??**
 - Resposta: **ABSTRAÇÃO!**
 - Abstrações são planejadas, para que novos comportamentos sejam incorporados através de novas classes derivadas

PPOO – Aberto-Fechado

- “*Programe para uma 'interface', não para uma 'implementação'*” (GoF – gang of four, 1994)
- **Acoplamento com classe concreta (ruim)**
 - Amarrado ao contrato e à implementação
 - Difícil estender sem impacto
- **Acoplamento com abstração (bom)**
 - Amarrado apenas ao contrato da classe abstrata ou interface
 - Fácil inserir novas implementações

PPOO – Aberto-Fechado

- Exemplo:



PPOO – Aberto/Fechado

- **Fechamento Estratégico**

- Não é possível atingir 100% de “fechamento”
- Não é possível (nem viável) querer prever tudo
 - Ponto de Variação X Ponto de Evolução

- **Exercício rápido**

- Novo requisito para a figura anterior
 - *“Círculos devem ser desenhados antes de Quadrados”*
- Discussão em pares
 - O que mudou?
 - Como lidar com essa mudança? Qual a melhor abstração?

PPOO – Aberto/Fechado

- **Exercício rápido**

- Novo requisito para a figura anterior
 - “*Círculos devem ser desenhados antes de Quadrados*”
- Discussão em pares
 - O que mudou?
 - Como lidar com essa mudança? Qual a melhor abstração?

ORDENAR A LISTA DE FORMAS

- Mesmo assim cada adição de nova forma vai gerar mudanças no método de comparação de todas as formas.
- Não é possível fechar sempre

Exercício Rápido

```
public interface Shape : Icomparable {  
  
    void Draw();  
  
}  
  
public void DrawAllShapes(ArrayList shapes){  
  
    shapes.Sort();  
  
    foreach(Shape shape in shapes)  
  
        shape.Draw();  
  
}
```

```
public class ShapeComparer : IComparer{  
  
    private static Hashtable priorities = new Hashtable();  
  
    static ShapeComparer(){  
  
        priorities.Add(typeof(Circle), 1);  
  
        priorities.Add(typeof(Square), 2);  
  
    }  
  
    private int PriorityFor(Type type){  
  
        if(priorities.Contains(type)) return (int)priorities[type];  
  
        else return 0;  
  
    }  
  
    public int Compare(object o1, object o2){  
  
        int priority1 = PriorityFor(o1.GetType());  
  
        int priority2 = PriorityFor(o2.GetType());  
  
        return priority1.CompareTo(priority2);  
  
    }  
}
```

Exercício Rápido

```
public interface Shape : Icomparable {  
  
    void Draw();  
  
}  
  
public void DrawAllShapes(ArrayList shapes){  
  
    shapes.Sort();  
  
    foreach(Shape shape in shapes)  
  
        shape.Draw();  
  
}
```

```
public class ShapeComparer : IComparer{  
  
    private static Hashtable priorities = new Hashtable();  
  
    static ShapeComparer(){  
  
        priorities.Add(typeof(Circle), 1);  
  
        priorities.Add(typeof(Square), 2);  
  
    }  
  
    private int PriorityFor(Type type){  
  
        if(priorities.Contains(type)) return (int)priorities[type];  
  
        else return 0;  
  
    }  
  
    public int Compare(object o1, object o2){  
  
        int priority1 = PriorityFor(o1.GetType());  
  
        int priority2 = PriorityFor(o2.GetType());  
  
        return priority1.CompareTo(priority2);  
  
    }  
}
```

PPOO – Aberto-Fechado

VOCÊ!!!

- **Projetista de software deve:**

- sempre considerar todos os Pontos de Variação (ou seja, o que já afeta o sistema) já elicitados

- **Ex.: Círculos, Quadrados**

- baseado no conhecimento do NEGÓCIO, considerar prováveis mudanças

- **Ex.: Pessoa jurídica (requisito atual só pede Física)**

e evitar as improváveis

- **Ex.: Lançar dois consumidores em mesma Nota Fiscal**

- baseado no conhecimento de COMPUTAÇÃO, considerar variações de ambiente e tecnologia

- **Ex.: Fornecedor do SGBD, Formato de dados, Endereços de recursos**

PPOO – Aberto-Fechado

VOCÊ!!!

- **Projetista de software deve:**

- sempre considerar todos os Pontos de Variação (ou seja, o que já afeta o sistema) já elicitados

- **Ex.: Círculos, Quadrados**

- baseado no conhecimento do NEGÓCIO, considerar prováveis mudanças

- **Ex.: Pessoa jurídica (requisito atual só pede Física)**

e evitar as improváveis

- **Ex.: Lançar dois consumidores em mesma Nota Fiscal**

- baseado no conhecimento de COMPUTAÇÃO, considerar variações de ambiente e tecnologia

- **Ex.: Fornecedor do SGBD, Formato de dados, Endereços de recursos**

Conclusão

- **Princípios de Robert Martin nos mostram que**
 - Gerenciar dependências é uma forma de garantir a longevidade do *design* do software
- **Responsabilidade Única (SRP)** nos mostra que
 - Coesão alta limita impacto de mudança
- **Aberto-Fechado (OCP)** nos mostra que
 - Manutenabilidade não é coisa do acaso, deve ser planejada